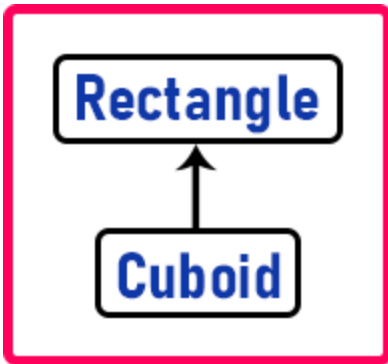


Generalization and Specialization in C# with Examples

In this article, I am going to discuss **Generalization and Specialization in C#** with Examples. Please read our previous article where we discussed [IS-A and HAS-A Relationships in C#](#) with Examples. This is a very interesting topic that is generalization vs specialization. This is related to inheritance. Let us understand what you mean by Generalization and Specialization in C# with Examples.

Specialization in C#

Let's say we have a class called Rectangle. Then, can we create the object of the Rectangle class? Yes, we can create objects of Rectangle and call the functions of the Rectangle class. Now, we have a class called Cuboid which is inherited from the Rectangle class.



Now, for the Cuboid class also we can create objects and call the functions of Cuboid and Rectangle. These two things i.e. **Rectangle and Cuboid really exist in the world**. Can you show me a Rectangle? Yes, you can show me some shapes that are rectangular or paper that is rectangular. So, rectangles exist in the real world. And what about Cuboid? Yes, it also exists in the real world. Any box shape thing is a cuboid.

So, these two things exist in the real world. Now, if I say, which one is existing first in our example? Then the answer will be Rectangle. And from the Rectangle class, we have derived the Cuboid class. So, the Rectangle class is already existing and from the Rectangle class, we have defined a new class with some extra features i.e. we have a specialized class that is Cuboid. This is a specialization in C#.

Example to Understand Specialization in C#:

In the below example first, we created the Rectangle class and from the Rectangle class, we have created a specialized class with some extra features called the Cuboid class. So, Cuboid is a specialization class of Rectangle.

```
using System;
namespace IsAHasADemo
{
    public class Program
    {
        static void Main()
        {
            Cuboid cuboid = new Cuboid(3, 5, 7);
            Console.WriteLine($"Volume is : {cuboid.Volume()}");
            Console.WriteLine($"Area is : {cuboid.Area()}");
        }
    }
}
```

```

        Console.WriteLine($"Perimeter is : {cuboid.Perimeter()}");

        Console.ReadKey();
    }
}
class Rectangle
{
    public int Length;
    public int Breadth;
    public int Area()
    {
        return Length * Breadth;
    }
    public int Perimeter()
    {
        return 2 * (Length + Breadth);
    }
}
class Cuboid : Rectangle
{
    public int Height;
    public Cuboid(int l, int b, int h)
    {
        Length = l;
        Breadth = b;
        Height = h;
    }
    public int Volume()
    {
        return Length * Breadth * Height;
    }
}
}

```

Output:

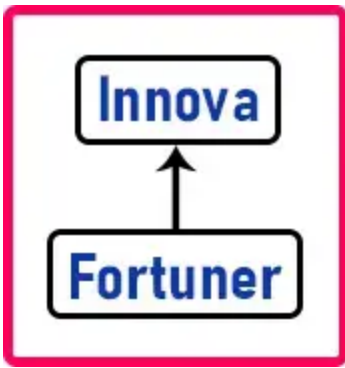
```

Volume is : 105
Area is : 15
Perimeter is : 16

```

Another Example to Understand specialization in C#:

Let us take another example of cars. We have a class Innova, Innova car from Toyota company. Then there is another class called Fortuner.

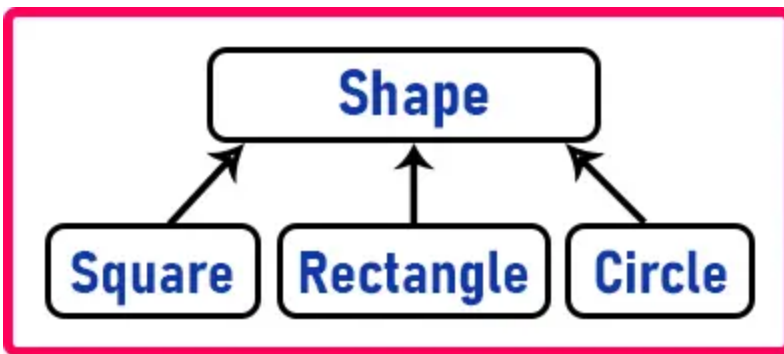


If you know about these then you should know that first Innova was launched by Toyota and then a new car called Fortuner was launched which is an extension of Innova. As a common man if you see Fortuner as an extension of Innova then it is having same seating capacity but with a lot of extra features compared to Innova. So, it means that the Fortuner car is an extension of the Innova car.

Innova is a base class and Fortuner is a derived class. Innova was already existing and Fortuner is the specialized form of Innova. This is specialization. **Something is already existing and then we are deriving something from that existing one by adding some extra features.** In the real world, we can take a lot of examples of specialization.

Generalization in C#

Let's say we have 3 classes that are Square, Rectangle, and Circle. Then for all of these three classes, we are defining one class that is Shape. And Square, Rectangle, and Circle are inherited from the Shape class. For a better understanding, please have a look at the following diagram.



So, among these classes which one is existing first? We know that all the shapes i.e. square, rectangle and circle really exist in the real world. So, we are calling them shapes.

Do you think Shape really exists in the real world? No. It is just a virtual term. If I asked you to show me some shapes, then you'll be drawing a rectangle, a circle, a square, a Rhombus, etc. but you cannot show me just a shape. So, Shape is a term or generic term, or generalized term which does not exist in the real world.

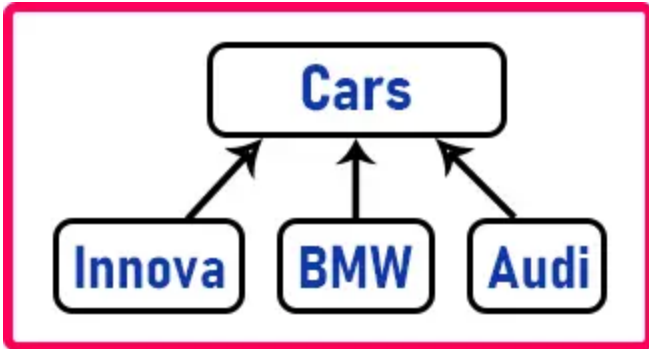
These child classes which are already existing in the real world and to bring them on a common platform, we have given the word Shape which is a logical term or virtual term, it is not a real term, it does not really exist in the real world.

Can you find the area or perimeter of a rectangle? Yes. Can you find the area or perimeter of the square? Yes. Can you find the area or perimeter of the circle? Yes. Does every shape have an area and perimeter? Yes.

This is a common feature, so we have brought it into shape. Now can you calculate the area and perimeter of the Shape? No, you can't. There are no dimensions for shape. It's a generalized term. So, this is an example of Generalization in C#.

Another Example to understand Generalization:

Suppose you have Innova, BMW, and Audi cars. So, you can start, stop, accelerate, and change gears in any of these. So, what these are all? These all are cars. Innova, BMW, and Audi are physically available in the real world. All these are nothing but cars.



So, they all are inheriting from Cars. What can you do in a car? You can drive, start, stop, change gears, etc. all these features are available in these cars. Then what about Cars? Does it really exist? No, it is a general term. Just a general term that is the logical definition of a group of things.

So, we have given a superclass to a group of classes for refereeing something. It is just like if I am looking at Innova so I can say it is a car. So, for any 4-wheelers we use the term, Car.

We are using this term generalization for refereeing them. We don't have to know the brand name or the product name perfectly. When it looks like a car, we can say that's a car. So, in the real world or in daily life we define such general terms.

Example to Understand Generalization in C#

```
using System;
namespace IsAHasADemo
{
    public class Program
    {
        static void Main()
        {
            Innova innova = new Innova();
            innova.Start();
            innova.Stop();
            BMW bmw = new BMW();
            bmw.Start();
            bmw.Stop();
            Console.ReadKey();
        }
    }
}
```

```

interface ICar
{
    void Start();
    void Stop();
}
public class Innova : ICar
{
    public void Start()
    {
        Console.WriteLine($"Innova Start");
    }
    public void Stop()
    {
        Console.WriteLine($"Innova Stop");
    }
}
public class BMW : ICar
{
    public void Start()
    {
        Console.WriteLine($"BMW Start");
    }
    public void Stop()
    {
        Console.WriteLine($"BMW Stop");
    }
}
}

```

Output:

```

Innova Start
Innova Stop
BMW Start
BMW Stop

```

Summary of Generalization and Specialization in C#

So, the point that you need to remember is both generalization and specialization achieve using only inheritance. We have seen four examples in which 2 are of specialization and 2 are of generalization. In specialization, the parent was existing and the child was defined later. In generalization, the child class was existing then we define the base class. So, specialization is a top-down approach and generalization is a bottom-up approach.

In specialization, the base class has something to give to the child class whereas, in generalization, the base class doesn't have anything to give to their child classes. Just their purpose is to group them together so that we can easily manage all those things.

The purpose of generalization is to achieve polymorphism and the purpose of specialization is to share its features with its child classes. What is polymorphism and how to implement polymorphism will discuss in our

coming articles?

So, there are two purposes of inheritance. One is to share features with child classes and the second one is to achieve polymorphism. So here I have given you a clear idea of what is a specialization and what is a generalization.

In the next article, I am going to discuss the [Interface in C#](#) with Examples. Here, in this article, I try to explain **Generalization and Specialization in C#** with Examples and I hope you enjoy this Generalization and Specialization in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.